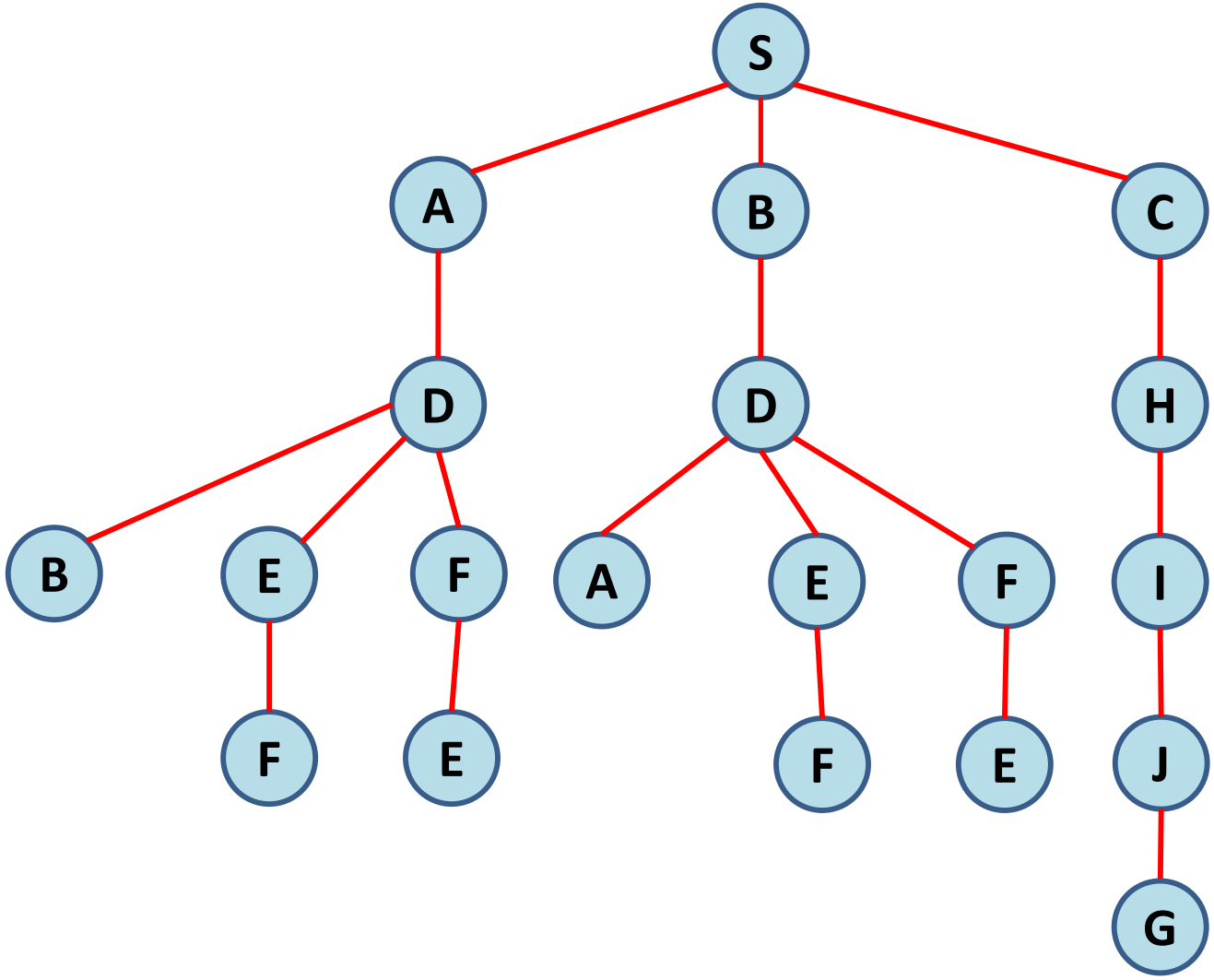
TUTORIAL#1: SEARCH

Search Tree



The heuristic distance estimates to the goal are:

S = 10, A = 5, B = 8, C = 6, D = 7, E = 6, F = 5, H = 3, I = 2, J = 1, G = 0

Let’s apply the following algorithm to solve the problem:

Depth-First Search without the Visited List

SEARCH

Simple Search Algorithm

A Search Node is a path from some state X to the start state
e.g. (X B A S)

The state of a search node is the most recent state of the path
e.g. X

Let Q be a list of search nodes
e.g. (X B A S) (C B A S) …)

Let S be the start state.

1. Initialise Q with search node (S) as only entry.
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q.
7. Go to Step 2.

Critical Decisions:

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

2) DFS: Pick **first** element of Q; Add path extensions to **front** of Q.

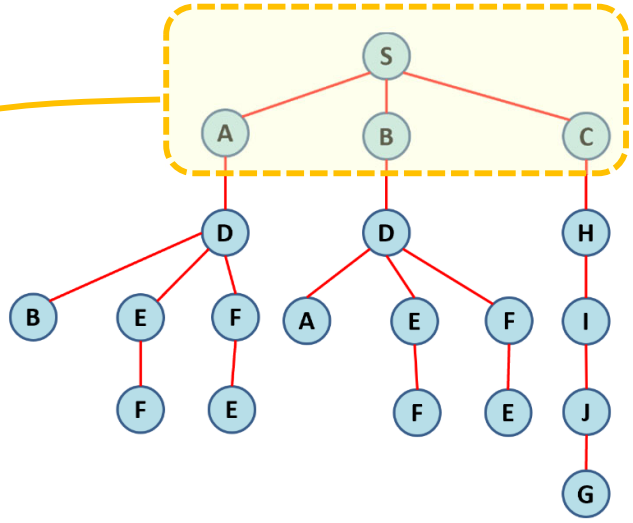
6) DFS: Nodes are pulled off the Q and inserted into the Q using a **stack**.

SEARCH

Depth-First without the Visited List

Step	Dequeued	Q
0		(S)
1	(S)	(AS) (BS) (CS)

Search Tree



Pick **first** element of Q; Add path extensions to **front** of Q.

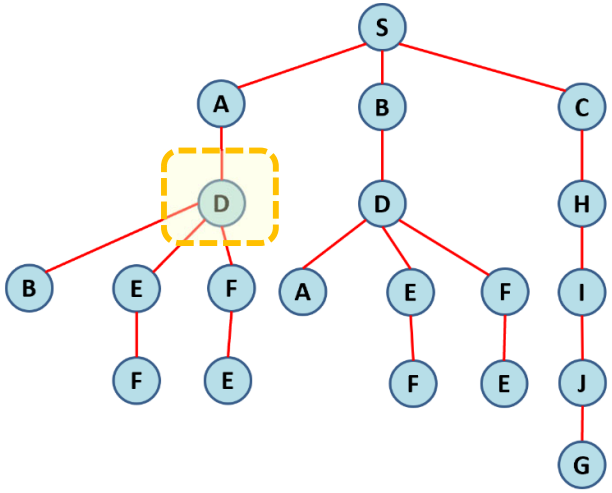
In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

SEARCH

Depth-First without the Visited List

Step	Dequeued	Q
0		(S)
1	(S)	(AS) (BS) (CS)
2	(AS)	(DAS) (BS) (CS)

Search Tree



Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

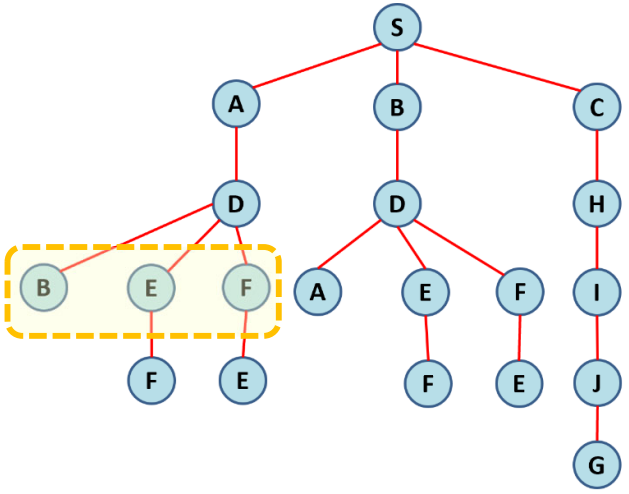
SEARCH

Depth-First without the Visited List

Step	Dequeued	Q
0		(S)
1	(S)	(AS) (BS) (CS)
2	(AS)	(DAS) (BS) (CS)
3	(DAS)	(BDAS)(EDAS) (FDAS) (BS)(CS)
4	(BDAS)	(EDAS) (FDAS) (BS) (CS)
5	(EDAS)	(FEDAS) (FDAS) (BS) (CS)

continued…

Search Tree



(BDAS) is added as the algorithm has no way of telling whether it has already a node in the Q whose state is also B

Pick **first** element of Q; Add path extensions to **front** of Q.

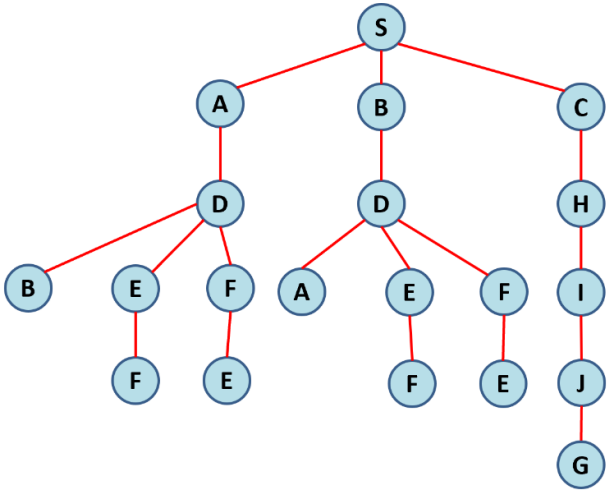
In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

SEARCH

Depth-First without the Visited List

Step	Dequeued	Q
6	(FEDAS)	(FDAS)(BS) (CS) dead end
7	(FDAS)	(EFDAS) (BS) (CS)
8	(EFDAS)	(BS)(CS)
9	(BS)	(DBS) (CS)
10	(DBS)	(ADBS)(EDBS) (FDBS)(CS)
11	(ADBS)	(EDBS) (FDBS)(CS)
…		

Search Tree



Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

2): DFS without Visited List

Step	Dequened	Enqueued
0		(S)
1	(S)	(AS)(BS)(CS)
2	(AS)	(DAS)(BS)(CS)
3	(DAS)	(BDAS)(EDAS)(FDAS)(BS)(CS)
4	(BDAS)	(EDAS)(FDAS)(BS)(CS)
5	(EDAS)	(FEDAS)(FDAS)(BS)(CS)
6	(FEDAS)	(EDAS)(BS)(CS)
7	(FDAS)	(EFDAS)(BS)(CS)
8	(EFDAS)	(BS)(CS)
9	(BS)	(DBS)(CS)
10	(DBS)	(ADBS)(EDBS)(FDBS)(CS)
11	(ADBS)	(EDBS)(FDBS)(CS)
12	(EDBS)	(FEDBS)(FDBS)(CS)
13	(FEDBS)	(FDBS)(CS)
14	(FDBS)	(EFDBS)(CS)
15	(EFDBS)	(CS)
16	(CS)	(HCS)
17	(HCS)	(IHCS)
18	(IHCS)	(JIHCS)
19	(JIHCS)	(GJIHCS)
20	(GJIHCS)	

Sequence of State expansion: S-A-D-B-E-F-F-E-B-D-A-E-F-F-E-C-H-I-J-G

SEARCH

Depth-First without the Visited List

Characterisation:

Sequence of state expansions = ?

Path = ?

Max Q Length = ?

Number of state expansions = ?

SEARCH

Depth-First without the Visited List

Characterisation:

Sequence of state expansions = S-A-D-B-E-F-F-E-B-D-A-E-F-F-E-C-H-I-J-G

Path = S-C-H-I-J-G

Max Q Length = 5

Number of state expansions = 20

Depth-First Search without the Visited List expanded more states (20 vs. 11) than the one using the visited list. Moreover, it used more memory space (5 vs. 4). In this simple example, we can see the consequence of not using the visited list. In the programming assignment, we will see much bigger differences in their performance.